MERN Stack Bootcamp – Hands-On Practice
Instructor: Kashif Raza | LinkedIn
Duration: 3–4 hours (self-paced)
Difficulty: Beginner → Intermediate

📋 Lab Overview

🎯 Learning Objectives

By completing this lab, you will:

1. ✅ Implement all major React hooks in realistic scenarios
2. ✅ Build a feature-complete Task Manager with persistence, filtering, and optimization
3. ✅ Extract reusable logic into custom hooks
4. ✅ Apply performance optimization patterns correctly
5. ✅ Debug common hook-related issues using React DevTools

🧰 Prerequisites

- Node.js 18+ and npm/yarn installed
- Basic React knowledge (components, props, JSX)
- Familiarity with the lecture content on React Hooks
- Code editor (VS Code recommended) with ESLint + React hooks plugin

📦 Project Setup

```
# 1. Create project with Vite (faster than CRA)
npm create vite@latest hooks-lab -- --template react
cd hooks-lab

# 2. Install dependencies
npm install

# 3. Install ESLint plugin for hooks
npm install eslint-plugin-react-hooks --save-dev

# 4. Start dev server
npm run dev
```

🔧 ESLint Configuration (`.eslintrc.js`)

```
module.exports = {
  plugins: ['react-hooks'],
  rules: {
    'react-hooks/rules-of-hooks': 'error',
    'react-hooks/exhaustive-deps': 'warn'
  }
};
```

📁 Final Project Structure

```
hooks-lab/
├── src/
```

```
│   ├── components/
│   │   ├── TaskForm.jsx
│   │   ├── TaskList.jsx
│   │   ├── TaskItem.jsx
│   │   ├── FilterBar.jsx
│   │   └── StatsPanel.jsx
│   ├── contexts/
│   │   └── TaskContext.jsx
│   ├── hooks/
│   │   ├── useLocalStorage.js
│   │   ├── useDebounce.js
│   │   └── useTaskManager.js
│   ├── utils/
│   │   └── helpers.js
│   ├── App.jsx
│   └── main.jsx
├── package.json
└── README.md
```

🧭 Lab Roadmap

Phase	Focus	Hooks Practiced	Time
Phase 1	Basic State & Effects	`useState`, `useEffect`	45 min
Phase 2	Context & Persistence	`useContext`, `useEffect`	45 min
Phase 3	Complex State Logic	`useReducer`	30 min
Phase 4	Performance Optimization	`useCallback`, `useMemo`	45 min
Phase 5	Custom Hooks & Refs	`useRef`, Custom Hooks	30 min
Phase 6	Polish & Challenges	All hooks	30 min

🚀 Phase 1: Basic State & Effects (`useState`, `useEffect`)

🎯 Goal

Build a functional TaskList with add, toggle, and delete capabilities using local state.

📝 Tasks

Task 1.1: Setup TaskForm with `useState` (15 mins)

File: `src/components/TaskForm.jsx`

```jsx
// STARTER CODE
import { useState } from 'react';

export default function TaskForm({ onAddTask }) {
  const [title, setTitle] = useState('');

  const handleSubmit = (e) => {
    e.preventDefault();
    if (!title.trim()) return;

    // TODO: Call onAddTask with new task object
    // Task shape: { id: Date.now(), title, completed: false, createdAt: new Date() }
```

```
    setTitle(''); // Reset input
  };

  return (
    <form onSubmit={handleSubmit} className="task-form">
      <input
        type="text"
        value={title}
        onChange={(e) => setTitle(e.target.value)}
        placeholder="What needs to be done?"
        className="task-input"
      />
      <button type="submit" className="btn-primary">Add Task</button>
    </form>
  );
}
```

✅ **Checklist:**

- [ ] Input value is controlled by `useState`
- [ ] Form prevents empty submissions
- [ ] Input clears after successful add
- [ ] New task has correct shape with unique `id`

Task 1.2: Build TaskList with `useState` (20 mins)

File: `src/components/TaskList.jsx`

```
// STARTER CODE
import { useState } from 'react';
import TaskItem from './TaskItem';

export default function TaskList() {
  const [tasks, setTasks] = useState([]);

  const addTask = (newTask) => {
    // TODO: Add new task to state immutably
  };

  const toggleTask = (id) => {
    // TODO: Toggle completed status by id
  };

  const deleteTask = (id) => {
    // TODO: Remove task by id
  };

  return (
    <div className="task-list">
      <TaskForm onAddTask={addTask} />

      {tasks.length === 0 ? (
        <p className="empty-state">No tasks yet. Add one above! ✨</p>
      ) : (
        <ul>
          {tasks.map(task => (
            <TaskItem
```

```
              key={task.id}
              task={task}
              onToggle={() => toggleTask(task.id)}
              onDelete={() => deleteTask(task.id)}
            />
          ))}
        </ul>
      )}
    </div>
  );
}
```

✅ **Checklist:**

- [ ] `addTask` uses immutable update: `setTasks(prev => [...prev, newTask])`
- [ ] `toggleTask` uses `map()` to update specific task
- [ ] `deleteTask` uses `filter()` to remove task
- [ ] Empty state displays when no tasks exist

Task 1.3: Persist Tasks with `useEffect` (10 mins)

File: `src/components/TaskList.jsx` (continue)

```
// Add this useEffect to TaskList component
useEffect(() => {
  // TODO: Load tasks from localStorage on mount
  // Use: JSON.parse(localStorage.getItem('tasks')) || []

  // TODO: Save tasks to localStorage whenever tasks array changes
  // Use: localStorage.setItem('tasks', JSON.stringify(tasks))
}, []); // What dependencies should go here?
```

⚠️ **Critical Thinking:**

- Why do we need TWO `useEffect` calls (or one with conditional logic)?
- What happens if we put `tasks` in the dependency array of the load effect?
- How do we handle JSON parse errors gracefully?

✅ **Checklist:**

- [ ] Tasks persist after page refresh
- [ ] No errors when localStorage is empty or corrupted
- [ ] `useEffect` dependencies are correctly specified

🧪 Phase 1 Verification

```
# Run app and test:
npm run dev

# Manual tests:
1. Add 3 tasks → refresh page → tasks should remain
2. Toggle a task → refresh → completed state persists
3. Delete a task → refresh → task stays deleted
4. Add empty task → should not submit
```

🔄 Phase 2: Context & Global State (`useContext`)

🎯 Goal

Eliminate prop drilling by moving task state to a shared Context.

📝 Tasks

Task 2.1: Create TaskContext (15 mins)

File: `src/contexts/TaskContext.jsx`

```
// STARTER CODE
import { createContext, useContext, useState, useEffect } from 'react';

const TaskContext = createContext(null);

export function TaskProvider({ children }) {
  const [tasks, setTasks] = useState([]);
  const [filter, setFilter] = useState('all'); // 'all' | 'active' | 'completed'

  // Load from localStorage on mount
  useEffect(() => {
    try {
      const saved = localStorage.getItem('tasks');
      if (saved) setTasks(JSON.parse(saved));
    } catch (e) {
      console.warn('Failed to load tasks:', e);
    }
  }, []);

  // Save to localStorage when tasks change
  useEffect(() => {
    try {
      localStorage.setItem('tasks', JSON.stringify(tasks));
    } catch (e) {
      console.warn('Failed to save tasks:', e);
    }
  }, [tasks]);

  // Action functions
  const addTask = (title) => {
    const newTask = {
      id: Date.now(),
      title,
      completed: false,
      createdAt: new Date().toISOString()
    };
    setTasks(prev => [...prev, newTask]);
  };

  const toggleTask = (id) => {
    setTasks(prev => prev.map(task =>
      task.id === id ? { ...task, completed: !task.completed } : task
    ));
  };

  const deleteTask = (id) => {
    setTasks(prev => prev.filter(task => task.id !== id));
  };
```

```
  const clearCompleted = () => {
    setTasks(prev => prev.filter(task => !task.completed));
  };

  // Compute filtered tasks
  const filteredTasks = tasks.filter(task => {
    if (filter === 'active') return !task.completed;
    if (filter === 'completed') return task.completed;
    return true; // 'all'
  });

  // Stats for dashboard
  const stats = {
    total: tasks.length,
    active: tasks.filter(t => !t.completed).length,
    completed: tasks.filter(t => t.completed).length
  };

  const value = {
    tasks: filteredTasks,
    stats,
    filter,
    setFilter,
    addTask,
    toggleTask,
    deleteTask,
    clearCompleted
  };

  return (
    <TaskContext.Provider value={value}>
      {children}
    </TaskContext.Provider>
  );
}

// Custom hook for easy consumption
export const useTasks = () => {
  const context = useContext(TaskContext);
  if (!context) {
    throw new Error('useTasks must be used within TaskProvider');
  }
  return context;
};
```

✅ **Checklist:**

- [ ] Context provides all required actions and state
- [ ] `useTasks` custom hook includes error handling
- [ ] localStorage operations are wrapped in try/catch
- [ ] Filter logic correctly computes `filteredTasks`

Task 2.2: Refactor Components to Use Context (20 mins)

File: `src/components/TaskForm.jsx` (updated)

```
import { useTasks } from '../contexts/TaskContext';

export default function TaskForm() {
  const { addTask } = useTasks(); // ✅ No more props!
  const [title, setTitle] = useState('');

  const handleSubmit = (e) => {
    e.preventDefault();
    if (!title.trim()) return;
    addTask(title); // ✅ Direct context call
    setTitle('');
  };
  // ... rest of component
}
```

File: `src/components/TaskList.jsx` (updated)

```
import { useTasks } from '../contexts/TaskContext';
import TaskItem from './TaskItem';
import FilterBar from './FilterBar';

export default function TaskList() {
  const { tasks, toggleTask, deleteTask, stats } = useTasks();

  return (
    <div className="task-list">
      <TaskForm />
      <FilterBar />  {/* New component */}

      {tasks.length === 0 ? (
        <p className="empty-state">No tasks match your filter. 🎯</p>
      ) : (
        <ul>
          {tasks.map(task => (
            <TaskItem
              key={task.id}
              task={task}
              onToggle={() => toggleTask(task.id)}
              onDelete={() => deleteTask(task.id)}
            />
          ))}
        </ul>
      )}

      <StatsPanel stats={stats} />  {/* New component */}
    </div>
  );
}
```

File: `src/components/FilterBar.jsx` (NEW)

```
import { useTasks } from '../contexts/TaskContext';

export default function FilterBar() {
  const { filter, setFilter, clearCompleted, stats } = useTasks();

  return (
    <div className="filter-bar">
      <div className="filter-buttons">
        {['all', 'active', 'completed'].map(f => (
```

```
          <button
            key={f}
            className={filter === f ? 'active' : ''}
            onClick={() => setFilter(f)}
          >
            {f.charAt(0).toUpperCase() + f.slice(1)}
          </button>
        ))}
      </div>

      {stats.completed > 0 && (
        <button onClick={clearCompleted} className="btn-secondary">
          Clear Completed ({stats.completed})
        </button>
      )}
    </div>
  );
}
```

✅ **Checklist:**

- [ ] All components consume context via `useTasks()`
- [ ] No prop drilling from `App.jsx` to children
- [ ] Filter buttons update UI correctly
- [ ] "Clear Completed" only shows when applicable

Task 2.3: Wrap App with Provider (10 mins)

File: `src/App.jsx`

```jsx
import { TaskProvider } from './contexts/TaskContext';
import TaskList from './components/TaskList';

export default function App() {
  return (
    <TaskProvider>
      <div className="app">
        <h1>🎯 Task Manager</h1>
        <TaskList />
      </div>
    </TaskProvider>
  );
}
```

✅ **Checklist:**

- [ ] App renders without context errors
- [ ] All features from Phase 1 still work
- [ ] Filter functionality works end-to-end

🧪 Phase 2 Verification

```
# Test context integration:
1. Add tasks → switch filter → correct tasks show
2. Complete 2 tasks → "Clear Completed" appears → click → tasks removed
```

```
3. Refresh page → filter state resets (expected) but tasks persist
4. Open React DevTools → Components → verify context value updates
```

⚙️ Phase 3: Complex State with `useReducer`

🎯 Goal

Refactor TaskContext to use `useReducer` for more predictable state transitions.

📝 Tasks

Task 3.1: Define Reducer Logic (15 mins)

File: `src/contexts/TaskContext.jsx` (refactor)

```jsx
// Replace useState with useReducer

// 1. Define initial state
const initialState = {
  tasks: [],
  filter: 'all'
};

// 2. Define reducer function
function taskReducer(state, action) {
  switch (action.type) {
    case 'LOAD_TASKS':
      return { ...state, tasks: action.payload };

    case 'ADD_TASK':
      return {
        ...state,
        tasks: [...state.tasks, action.payload]
      };

    case 'TOGGLE_TASK':
      return {
        ...state,
        tasks: state.tasks.map(task =>
          task.id === action.payload
            ? { ...task, completed: !task.completed }
            : task
        )
      };

    case 'DELETE_TASK':
      return {
        ...state,
        tasks: state.tasks.filter(task => task.id !== action.payload)
      };

    case 'CLEAR_COMPLETED':
      return {
        ...state,
        tasks: state.tasks.filter(task => !task.completed)
      };

    case 'SET_FILTER':
      return { ...state, filter: action.payload };
```

```
    case 'RESET':
      return initialState;

    default:
      throw new Error(`Unknown action: ${action.type}`);
  }
}

// 3. Update TaskProvider component
export function TaskProvider({ children }) {
  const [state, dispatch] = useReducer(taskReducer, initialState);

  // Load from localStorage
  useEffect(() => {
    try {
      const saved = localStorage.getItem('tasks');
      if (saved) {
        dispatch({ type: 'LOAD_TASKS', payload: JSON.parse(saved) });
      }
    } catch (e) {
      console.warn('Failed to load tasks:', e);
    }
  }, []);

  // Save to localStorage when tasks change
  useEffect(() => {
    try {
      localStorage.setItem('tasks', JSON.stringify(state.tasks));
    } catch (e) {
      console.warn('Failed to save tasks:', e);
    }
  }, [state.tasks]);

  // Action creators
  const actions = {
    addTask: (title) => {
      const newTask = {
        id: Date.now(),
        title,
        completed: false,
        createdAt: new Date().toISOString()
      };
      dispatch({ type: 'ADD_TASK', payload: newTask });
    },
    toggleTask: (id) => dispatch({ type: 'TOGGLE_TASK', payload: id }),
    deleteTask: (id) => dispatch({ type: 'DELETE_TASK', payload: id }),
    clearCompleted: () => dispatch({ type: 'CLEAR_COMPLETED' }),
    setFilter: (filter) => dispatch({ type: 'SET_FILTER', payload: filter })
  };

  // Compute derived values
  const filteredTasks = state.tasks.filter(task => {
    if (state.filter === 'active') return !task.completed;
    if (state.filter === 'completed') return task.completed;
    return true;
  });

  const stats = {
    total: state.tasks.length,
    active: state.tasks.filter(t => !t.completed).length,
    completed: state.tasks.filter(t => t.completed).length
```

```
  };

  const value = {
    tasks: filteredTasks,
    stats,
    filter: state.filter,
    ...actions
  };

  return (
    <TaskContext.Provider value={value}>
      {children}
    </TaskContext.Provider>
  );
}
```

✅ **Checklist:**

- [ ] All state updates go through `dispatch`
- [ ] Reducer handles all action types
- [ ] localStorage sync still works with new structure
- [ ] No regressions in UI behavior

Task 3.2: Add Undo Feature with Reducer (15 mins)

Challenge: Implement undo for delete operations.

```
// Add to taskReducer:
case 'DELETE_TASK': {
  const taskToDelete = state.tasks.find(t => t.id === action.payload);
  return {
    ...state,
    tasks: state.tasks.filter(task => task.id !== action.payload),
    lastDeleted: taskToDelete // Store for undo
  };
}

case 'UNDO_DELETE': {
  if (!state.lastDeleted) return state;
  return {
    ...state,
    tasks: [...state.tasks, state.lastDeleted],
    lastDeleted: null
  };
}

// Add to actions in TaskProvider:
undoDelete: () => dispatch({ type: 'UNDO_DELETE' })

// In FilterBar.jsx, add undo button:
{state.lastDeleted && (
  <button onClick={undoDelete} className="btn-undo">
    Undo: "{state.lastDeleted.title}"
  </button>
)}
```

✅ **Checklist:**

- [ ] Deleting a task stores it in `lastDeleted`
- [ ] Undo button appears after delete
- [ ] Clicking undo restores the task
- [ ] Undo button disappears after use or new action

🧪 Phase 3 Verification

```
# Test reducer features:
1. Add → toggle → delete tasks → verify state transitions in React DevTools
2. Delete a task → click "Undo" → task reappears
3. Refresh after undo → task stays deleted (localStorage only saves tasks, not lastDeleted)
4. Test edge case: delete same task twice → undo should only restore once
```

⚡ Phase 4: Performance Optimization (`useCallback`, `useMemo`)

🎯 Goal

Optimize re-renders using memoization hooks—correctly and measurably.

📝 Tasks

Task 4.1: Memoize Context Value (15 mins)

Problem: Without memoization, all consumers re-render whenever ANY context value changes.

File: `src/contexts/TaskContext.jsx` (update)

```jsx
import { useMemo } from 'react';

// Inside TaskProvider:
const value = useMemo(() => ({
  tasks: filteredTasks,
  stats,
  filter: state.filter,
  addTask: actions.addTask,
  toggleTask: actions.toggleTask,
  deleteTask: actions.deleteTask,
  clearCompleted: actions.clearCompleted,
  setFilter: actions.setFilter,
  undoDelete: actions.undoDelete
}), [
  filteredTasks,
  stats,
  state.filter,
  actions.addTask,
  actions.toggleTask,
  actions.deleteTask,
  actions.clearCompleted,
  actions.setFilter,
  actions.undoDelete
]);

// But wait! actions are recreated every render...
// Fix: Memoize action creators too OR include dependencies
```

Better Approach: Memoize actions with `useCallback`

```
// Memoize action creators
const addTask = useCallback((title) => {
  const newTask = { /* ... */ };
  dispatch({ type: 'ADD_TASK', payload: newTask });
}, []); // dispatch is stable

const toggleTask = useCallback((id) => {
  dispatch({ type: 'TOGGLE_TASK', payload: id });
}, []);

// ... memoize other actions

// Now value dependencies are stable
const value = useMemo(() => ({
  tasks: filteredTasks,
  stats,
  filter: state.filter,
  addTask,
  toggleTask,
  // ... other memoized actions
}), [filteredTasks, stats, state.filter, addTask, toggleTask, /* ... */]);
```

✅ **Checklist:**

- [ ] Context value is wrapped in `useMemo`
- [ ] Action creators are wrapped in `useCallback`
- [ ] Dependency arrays are complete (ESLint clean)
- [ ] React DevTools Profiler shows fewer re-renders

Task 4.2: Optimize TaskItem with `React.memo` (15 mins)

File: `src/components/TaskItem.jsx`

```
import { memo, useCallback } from 'react';

// Wrap component with memo
const TaskItem = memo(function TaskItem({ task, onToggle, onDelete }) {
  // Use useCallback for inline handlers if needed
  const handleToggle = useCallback(() => {
    onToggle(task.id);
  }, [task.id, onToggle]);

  return (
    <li className={`task-item ${task.completed ? 'completed' : ''}`}>
      <input
        type="checkbox"
        checked={task.completed}
        onChange={handleToggle}
      />
      <span className="task-title">{task.title}</span>
      <button onClick={() => onDelete(task.id)} className="btn-delete">
        ×
      </button>
    </li>
  );
});
```

```
export default TaskItem;
```

✅ **Checklist:**

- [ ] `TaskItem` is wrapped with `memo()`
- [ ] Handlers use `useCallback` with correct dependencies
- [ ] Component only re-renders when its props actually change
- [ ] Test with React DevTools → Highlight updates when toggling OTHER tasks

---

Task 4.3: Memoize Expensive Calculations (15 mins)

Scenario: Add a "Search" feature that filters tasks by title (case-insensitive).

File: `src/contexts/TaskContext.jsx` (add search)

```
// Add to state:
const initialState = {
  tasks: [],
  filter: 'all',
  searchQuery: '' // NEW
};

// Add reducer case:
case 'SET_SEARCH':
  return { ...state, searchQuery: action.payload };

// Memoize the expensive filter+search computation
const filteredTasks = useMemo(() => {
  console.log('🔄 Filtering tasks...'); // Log to see when this runs

  let result = state.tasks;

  // Apply status filter
  if (state.filter === 'active') {
    result = result.filter(t => !t.completed);
  } else if (state.filter === 'completed') {
    result = result.filter(t => t.completed);
  }

  // Apply search (expensive on large lists)
  if (state.searchQuery.trim()) {
    const query = state.searchQuery.toLowerCase();
    result = result.filter(task =>
      task.title.toLowerCase().includes(query)
    );
  }

  return result;
}, [state.tasks, state.filter, state.searchQuery]); // Only recompute when these change
```

✅ **Checklist:**

- [ ] Search input updates `searchQuery` via context
- [ ] `filteredTasks` uses `useMemo` with correct dependencies
- [ ] Console log shows filtering only runs when needed
- [ ] Search works with status filters combined

🧪 Phase 4 Verification

```
# Profile performance:
1. Open React DevTools → Profiler tab
2. Record interaction: add 50 tasks (use loop in console)
3. Toggle a task → check which components re-rendered
4. Type in search → verify filteredTasks doesn't recompute on every keystroke (debounce next!)
5. Compare before/after memoization: fewer "wasted" renders
```

🪝 Phase 5: Custom Hooks & `useRef`

🎯 Goal

Extract reusable logic and manage imperative features with refs.

📝 Tasks

Task 5.1: Create `useLocalStorage` Hook (15 mins)

File: `src/hooks/useLocalStorage.js`

```
import { useState, useEffect } from 'react';

export function useLocalStorage(key, initialValue) {
  // Get initial value from localStorage or fallback
  const [storedValue, setStoredValue] = useState(() => {
    if (typeof window === 'undefined') return initialValue;

    try {
      const item = localStorage.getItem(key);
      return item ? JSON.parse(item) : initialValue;
    } catch (error) {
      console.warn(`Error reading localStorage key "${key}":`, error);
      return initialValue;
    }
  });

  // Update localStorage when value changes
  useEffect(() => {
    try {
      if (storedValue === undefined) {
        localStorage.removeItem(key);
      } else {
        localStorage.setItem(key, JSON.stringify(storedValue));
      }
    } catch (error) {
      console.warn(`Error setting localStorage key "${key}":`, error);
    }
  }, [key, storedValue]);

  // Return state + setter (with function update support)
  const setValue = (value) => {
    try {
      const valueToStore = value instanceof Function
        ? value(storedValue)
        : value;
      setStoredValue(valueToStore);
```

```
    } catch (error) {
      console.warn(`Error updating localStorage key "${key}":`, error);
    }
  };

  return [storedValue, setValue];
}
```

✅ **Checklist:**

- [ ] Hook handles SSR safely (`typeof window` check)
- [ ] JSON parse/stringify errors are caught
- [ ] Supports functional updates like `useState`
- [ ] Returns array `[value, setValue]` for destructuring

Task 5.2: Refactor Context to Use `useLocalStorage` (10 mins)

File: `src/contexts/TaskContext.jsx` (simplify)

```
import { useLocalStorage } from '../hooks/useLocalStorage';

export function TaskProvider({ children }) {
  // Replace useState + useEffect with custom hook!
  const [tasks, setTasks] = useLocalStorage('tasks', []);
  const [filter, setFilter] = useLocalStorage('task-filter', 'all');

  // Rest of reducer logic stays the same, but now persistence is automatic
  // ...
}
```

✅ **Checklist:**

- [ ] localStorage logic removed from TaskProvider
- [ ] Tasks and filter still persist correctly
- [ ] Code is cleaner and more testable

Task 5.3: Add Auto-Focus with `useRef` (15 mins)

File: `src/components/TaskForm.jsx` (enhance)

```
import { useRef, useEffect } from 'react';
import { useTasks } from '../contexts/TaskContext';

export default function TaskForm() {
  const { addTask } = useTasks();
  const [title, setTitle] = useState('');
  const inputRef = useRef(null);

  // Auto-focus input on mount
  useEffect(() => {
    inputRef.current?.focus();
  }, []);

  // Focus input after adding task (better UX)
  useEffect(() => {
```

```
    if (title === '') {
      inputRef.current?.focus();
    }
  }, [title]);

  const handleSubmit = (e) => {
    e.preventDefault();
    if (!title.trim()) return;
    addTask(title);
    setTitle('');
    // Focus is handled by useEffect above
  };

  return (
    <form onSubmit={handleSubmit} className="task-form">
      <input
        ref={inputRef} // ✅ Attach ref
        type="text"
        value={title}
        onChange={(e) => setTitle(e.target.value)}
        placeholder="What needs to be done?"
        className="task-input"
        aria-label="New task title"
      />
      <button type="submit" className="btn-primary">Add Task</button>
    </form>
  );
}
```

✅ **Checklist:**

- [ ] Input auto-focuses on page load
- [ ] Input re-focuses after adding a task
- [ ] `useRef` is used correctly (no re-renders on ref change)
- [ ] Accessibility: input has `aria-label`

🧪 Phase 5 Verification

```
# Test custom hooks and refs:
1. Delete localStorage manually → refresh → app initializes with defaults
2. Add task → verify focus behavior works smoothly
3. Test useLocalStorage hook in isolation (create test component)
4. Verify no console errors from localStorage operations
```

🏆 Phase 6: Polish & Challenges

🎯 Goal

Add advanced features and optimize the final product.

📝 Tasks

Task 6.1: Debounce Search Input (Bonus) (15 mins)

File: `src/hooks/useDebounce.js`

```
import { useState, useEffect } from 'react';

export function useDebounce(value, delay) {
  const [debouncedValue, setDebouncedValue] = useState(value);

  useEffect(() => {
    const handler = setTimeout(() => {
      setDebouncedValue(value);
    }, delay);

    return () => {
      clearTimeout(handler);
    };
  }, [value, delay]);

  return debouncedValue;
}
```

Usage in FilterBar.jsx:

```
import { useDebounce } from '../hooks/useDebounce';

export default function FilterBar() {
  const { searchQuery, setSearchQuery } = useTasks();
  const [inputValue, setInputValue] = useState(searchQuery);
  const debouncedSearch = useDebounce(inputValue, 300); // 300ms delay

  // Sync debounced value to context
  useEffect(() => {
    setSearchQuery(debouncedSearch);
  }, [debouncedSearch, setSearchQuery]);

  return (
    <div className="filter-bar">
      <input
        type="search"
        value={inputValue}
        onChange={(e) => setInputValue(e.target.value)}
        placeholder="Search tasks..."
        className="search-input"
      />
      {/* ... filter buttons */}
    </div>
  );
}
```

✅ **Checklist:**

- [ ] Search doesn't trigger on every keystroke
- [ ] Typing pauses → search executes after 300ms
- [ ] Cleanup prevents memory leaks (clearTimeout)

Task 6.2: Add Keyboard Shortcuts with `useRef` (Bonus) (15 mins)

File: `src/hooks/useKeyboardShortcut.js`

```
import { useEffect, useRef } from 'react';

export function useKeyboardShortcut(key, callback, dependencies = []) {
  const callbackRef = useRef(callback);

  // Update ref when callback changes (avoids stale closure)
  useEffect(() => {
    callbackRef.current = callback;
  }, [callback]);

  useEffect(() => {
    const handler = (event) => {
      // Ignore if typing in input/textarea
      if (['INPUT', 'TEXTAREA', 'SELECT'].includes(event.target.tagName)) {
        return;
      }

      if (event.key === key) {
        event.preventDefault();
        callbackRef.current(event);
      }
    };

    window.addEventListener('keydown', handler);
    return () => window.removeEventListener('keydown', handler);
  }, [key, ...dependencies]);
}
```

Usage in App.jsx:

```
import { useKeyboardShortcut } from './hooks/useKeyboardShortcut';
import { useTasks } from './contexts/TaskContext';

export default function App() {
  const { setFilter } = useTasks();

  // Press "a" to focus new task input (would need ref forwarding)
  // Press "1", "2", "3" to switch filters
  useKeyboardShortcut('1', () => setFilter('all'));
  useKeyboardShortcut('2', () => setFilter('active'));
  useKeyboardShortcut('3', () => setFilter('completed'));

  return (/* ... */);
}
```

✅ **Checklist:**

- [ ] Keyboard shortcuts work globally
- [ ] Shortcuts don't interfere with typing in inputs
- [ ] Event listeners are cleaned up properly
- [ ] `useRef` prevents stale callback issues

Task 6.3: Final Polish & Accessibility (15 mins)

Tasks:

- [ ] Add ARIA labels to all interactive elements

- [ ] Ensure color contrast meets WCAG AA standards
- [ ] Add loading states for async operations (if extended)
- [ ] Write JSDoc comments for all custom hooks
- [ ] Add a `README.md` with setup instructions and feature list

Example JSDoc:

```
/**
 * Persist state to localStorage with error handling
 * @template T
 * @param {string} key - localStorage key
 * @param {T} initialValue - Fallback value if key doesn't exist
 * @returns {[T, (value: T | ((prev: T) => T)) => void]} State tuple
 */
export function useLocalStorage(key, initialValue) { /* ... */ }
```

🧪 Final Verification Checklist

```
# Comprehensive testing:
✅ All Phase 1-5 features work end-to-end
✅ ESLint shows no hook-related warnings
✅ React DevTools Profiler: no unnecessary re-renders
✅ localStorage persists data correctly
✅ Keyboard shortcuts work (if implemented)
✅ App is accessible (keyboard nav, screen reader friendly)
✅ Code is documented with JSDoc for custom hooks
✅ README.md explains how to run and use the app
```

🐛 Troubleshooting Guide

Common Issues & Solutions

Issue	Likely Cause	Fix
Tasks don't persist	localStorage key mismatch or JSON error	Check key name; wrap parse in try/catch
Infinite re-renders	Missing/incorrect dependency array	Run ESLint; check `useEffect` deps
Stale state in callbacks	Using state directly instead of functional update	Use `setState(prev => ...)` pattern
Context not updating	Context value not memoized	Wrap value in `useMemo`
Custom hook not working	Not starting with `use` prefix	Rename to `useSomething`
Ref not focusing	Ref not attached or component not mounted	Check `ref` prop; add null check
Debounced search not updating	Missing dependency in `useEffect`	Add `debouncedValue` to deps

Debugging Tips

```
// 1. Log hook executions
useEffect(() => {
  console.log('🔄 TaskList re-rendered');
```

```
});

// 2. Use React DevTools
// - Components tab: inspect props/state/context
// - Profiler tab: record interactions, find wasted renders

// 3. Test custom hooks in isolation
// Create a test component:
function TestHook() {
  const [val, setVal] = useLocalStorage('test', 'default');
  return <button onClick={() => setVal('new')}>{val}</button>;
}

// 4. Enable StrictMode (already on in Vite dev)
// Double-invokes effects to catch side-effect bugs
```

📤 Submission Guidelines

What to Submit

1. **GitHub Repository** with:
   - Complete `src/` folder with all implemented features
   - `package.json` with all dependencies
   - `README.md` with:
     - Project description
     - Setup instructions (`npm install` , `npm run dev`)
     - List of hooks used and where
     - Screenshots of working app
   - `.gitignore` excluding `node_modules/` and `.env`
2. **Short Reflection** (in README or separate file):
   - Which hook was most challenging? Why?
   - One optimization you're proud of
   - What you'd add next (feature idea)

Evaluation Criteria

Criteria	Points
✅ All Phase 1-3 features working	30
✅ Performance optimizations (Phase 4)	20
✅ Custom hooks implemented correctly	20
✅ Code quality: clean, documented, ESLint clean	15
✅ Accessibility & UX polish	10
✅ Reflection & documentation	5
Total	**100**

Bonus Points (+10)

- 🎯 Implement undo/redo with history
- 🎯 Add drag-and-drop reordering (use `@dnd-kit`)
- 🎯 Write unit tests for custom hooks (Vitest + React Testing Library)
- 🎯 Deploy to Vercel/Netlify and share live link

📚 Resources & Next Steps

🔗 Helpful Links

- React Hooks API Reference
- React DevTools Guide
- useHooks.com - Production-ready custom hooks
- React Testing Library - Test hooks

🚀 Continue Learning

1. **Advanced Patterns**: `useImperativeHandle`, `useSyncExternalStore`
2. **State Management**: Compare Context + useReducer vs. Zustand/Jotai
3. **Performance**: Learn `React.memo`, code splitting, virtualization
4. **TypeScript**: Add types to all hooks and components

💬 Instructor Support

- Stuck? Post questions in our Discord `#hooks-lab` channel
- Office hours: [Link to calendar]
- Code review: Submit PR for feedback before final submission

---

> 💬 **Instructor Note:**
> This lab is designed to be challenging but achievable. If you get stuck:
>
> 1. Re-read the lecture section for that hook
> 2. Check the troubleshooting guide above
> 3. Ask for help—debugging is a core skill!
>
> Remember: The goal isn't perfection. It's understanding *when* and *why* to use each hook.
>
> I can't wait to see what you build. Share your final app on LinkedIn with `#MERNBootcamp` —I'll reshare the best ones!
>
> — Kashif Raza | Building robust systems, one hook at a time 🪝

Lab Manual Version: 1.0 | Last Updated: $(date)

© 2025 MERN Stack Bootcamp. For educational use only. Code examples are MIT licensed.